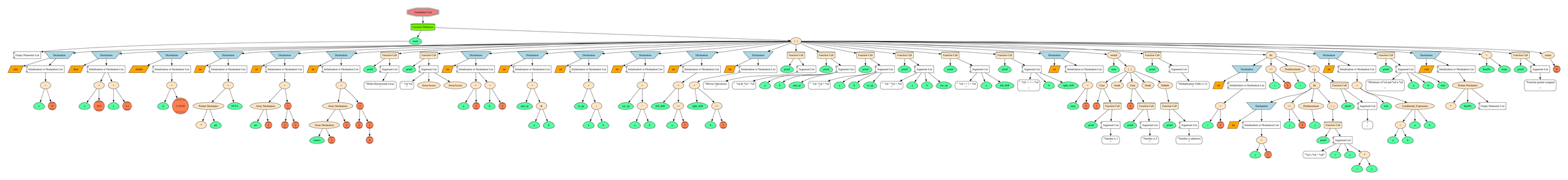

Translation Unit
Function Definition
main
{ }
Empty Parameter List
Declaration
char
Initialization or Declaration List
=
c
'A'
Declaration
float
Initialization or Declaration List
=
x
10.5
=
y
2.3
Declaration
double
Initialization or Declaration List
=
d
3.14159
Declaration
int
Initialization or Declaration List
=
Pointer Declarator
*
ptr
NULL
Declaration
int
Initialization or Declaration List
=
Array Declaration
arr
3
1
2
3
Declaration
int
Initialization or Declaration List
=
Array Declaration
Array Declaration
matrix
2
2
1
2
3
4
Function Call
printf
Argument List
"Multi-Dimensional Array: "
Function Call
printf
Argument List
" %d %d "
ArrayAccess
ArrayAccess
Declaration
int
Initialization or Declaration List
=
a
5
=
b
3
Declaration
int
Initialization or Declaration List
=
and_op
&
a
b
Declaration
int
Initialization or Declaration List
=
or_op
|
a
b
Declaration
int
Initialization or Declaration List
=
xor_op
^
a
b
Declaration
int
Initialization or Declaration List
=
left_shift
<<
a
1
=
right_shift
>>
b
1
Function Call
printf
Argument List
"Bitwise Operations: "
Function Call
printf
Argument List
" %d & %d = %d "
a
b
and_op
Function Call
printf
Argument List
" %d | %d = %d "
a
b
or_op
Function Call
printf
Argument List
" %d ^ %d = %d "
a
b
xor_op
Function Call
printf
Argument List
" %d << 1 = %d "
a
left_shift
Function Call
printf
Argument List
" %d >> 1 = %d "
b
right_shift
Declaration
int
Initialization or Declaration List
=
num
2
switch
num
{ }
Case
1
Function Call
printf
Argument List
"Number is 1 "
break
Case
2
Function Call
printf
Argument List
"Number is 2 "
break
Default
Function Call
printf
Argument List
"Number is unknown "
Function Call
printf
Argument List
"Multiplication Table (1-3): "
for
Declaration
int
Initialization or Declaration List
=
i
1
<=
i
3
PostIncrement
i
{ }
for
Declaration
int
Initialization or Declaration List
=
j
1
<=
j
3
PostIncrement
j
{ }
Function Call
printf
Argument List
"%d x %d = %dt"
i
j
*
i
j
Function Call
printf
Argument List
" "
Declaration
int
Initialization or Declaration List
=
min
Conditional_Expression
<
a
b
a
b
Function Call
printf
Argument List
"Minimum of %d and %d is %d "
a
b
min
Declaration
void
Initialization or Declaration List
Pointer Declarator
*
funcPtr
Empty Parameter List
=
funcPtr
main
Function Call
printf
Argument List
"Function pointer assigned "
return
0